09-PERFORMANCE

Bible Update: Part IX - Performance Optimizations

Status: DRAFT — Ready for review
New Section: Not in original Bible
Date: December 29, 2025
Primary Contributor: Gemini (Algorithms), Claude (Integration)
Target: <500ms to first token on Apple Silicon

Part IX: Performance Optimizations

9.1 The Latency Budget

Luna's voice interface requires **<500ms to first token**. That's the threshold where conversation feels natural rather than laggy.

The Budget Breakdown

```
┌──────────────────────────────────────────────────────────────────┐
│                    500ms LATENCY BUDGET                          │
│                                                                  │
│   STT Final Transcript ──────────────────────────── ~0ms         │
│   (already complete when we start)                               │
│                                                                  │
│    ┌──────────────────────────────────────────────────────┐      │
│    │ RETRIEVAL PHASE                              ~50ms   │      │
│    │                                                      │      │
│    │   Bloom filter check ──────────────── ~1ms           │      │
│    │   Query routing ────────────────────── ~2ms          │      │
│    │   FTS5 OR FAISS (not both) ─────────── ~30ms         │      │
│    │   Graph hop (rustworkx) ────────────── ~5ms          │      │
│    │   RRF merge (if needed) ────────────── ~2ms          │      │
│    │   Context assembly ─────────────────── ~10ms         │      │
│    │                                                      │      │
│    └──────────────────────────────────────────────────────┘      │
│                                                                  │
```

```
INFERENCE PHASE                          ~250ms

  KV cache load (pre-computed) ──────── ~5ms
  Context encoding ──────────────────── ~50ms
  First token generation ────────────── ~195ms

OUTPUT PHASE                             ~50ms

  Token to TTS ──────────────────────── ~10ms
  TTS processing ────────────────────── ~40ms

BUFFER ──────────────────────────────── ~150ms

TOTAL ───────────────────────────────── <500ms ✓
```

The Cheat: Speculative Execution

The budget above assumes we start retrieval *after* the user finishes speaking. But we don't have to wait:

```
User speaking: "What did we decide about the runtime..."
                      │
                      │ Partial transcript arrives
                      ▼
                ┌──────────────┐
                │ SPECULATIVE  │ ← Start retrieval NOW
                │  RETRIEVAL   │
                └──────────────┘
                      │
User finishes: "...runtime engine?"
                      │
                      │ Final transcript
                      ▼
                ┌──────────────┐
                │  VALIDATE    │ ← Is speculation still valid?
                │  SPECULATION │
                └──────────────┘
                      │
                      │ Yes (>0.85 similarity)
                      ▼
                ┌──────────────┐
                │  RETRIEVAL   │ ← Already done!
```

```
│   COMPLETE    │    Saved ~50ms
└───────────────┘
```

Result: Retrieval is often *already complete* when the user finishes speaking.

9.2 Speculative Execution

The Pattern

Start retrieval on partial STT transcripts. Validate when the final transcript arrives.

Implementation

```python
class SpeculativeRetriever:
    def __init__(self, matrix: MatrixActor, embedder: Embedder):
        self.matrix = matrix
        self.embedder = embedder
        self.speculative_task: Optional[asyncio.Task] = None
        self.speculative_query: Optional[str] = None
        self.speculative_embedding: Optional[np.ndarray] = None

    async def on_partial_transcript(self, text: str):
        """Start speculative retrieval on partial transcript."""
        # Cancel previous speculation if exists
        if self.speculative_task and not
         self.speculative_task.done():
            self.speculative_task.cancel()

        # Store for validation
        self.speculative_query = text
        self.speculative_embedding = await
         self.embedder.embed(text)

        # Fire speculative search
        self.speculative_task = asyncio.create_task(
            self._speculative_search(text)
        )

    async def on_final_transcript(self, text: str) ->
         RetrievalResults:
        """Validate speculation or fetch fresh."""

        if not self.speculative_task:
            # No speculation, fetch fresh
            return await self.matrix.search(text)

        # Wait for speculation to complete
        try:
            speculative_results = await asyncio.wait_for(
                self.speculative_task,
```

```
            timeout=0.1  # Don't wait long
        )
    except asyncio.TimeoutError:
        # Speculation not ready, fetch fresh
        return await self.matrix.search(text)

    # Validate speculation
    final_embedding = await self.embedder.embed(text)
    similarity = cosine_similarity(
        self.speculative_embedding,
        final_embedding
    )

    if similarity > 0.85:
        # Speculation valid, use cached results
        return speculative_results
    else:
        # Transcript changed significantly, fetch delta
        fresh_results = await self.matrix.search(text)
        return self._merge_results(speculative_results,
    fresh_results)

async def _speculative_search(self, text: str) ->
    RetrievalResults:
    """Execute the speculative search."""
    return await self.matrix.search(text)

def _merge_results(
    self,
    speculative: RetrievalResults,
    fresh: RetrievalResults
) -> RetrievalResults:
    """Merge speculative and fresh results using RRF."""
    return rrf_merge([speculative.nodes, fresh.nodes])
```

Validation Threshold

Similarity	Action
>0.85	Keep speculative results
0.70-0.85	Merge speculative + fresh
<0.70	Discard speculation, use fresh

Debouncing

Don't fire on every partial — debounce to avoid thrashing:

```
class DebouncedSpeculation:
    def __init__(self, delay_ms: int = 200):
        self.delay = delay_ms / 1000
        self.pending_task: Optional[asyncio.Task] = None
```

```
    async def on_partial(self, text: str, retriever:
        SpeculativeRetriever):
        """Debounced speculation trigger."""
        # Cancel pending
        if self.pending_task:
            self.pending_task.cancel()

        # Schedule new
        self.pending_task = asyncio.create_task(
            self._delayed_trigger(text, retriever)
        )

    async def _delayed_trigger(self, text: str, retriever:
        SpeculativeRetriever):
        await asyncio.sleep(self.delay)
        await retriever.on_partial_transcript(text)
```

9.3 KV Cache Management

The Problem

The Identity Buffer is ~2048 tokens. Re-encoding it every turn wastes ~50-100ms.

The Solution: Pre-computed KV Cache

```
from mlx_lm import cache_prompt, load_kv_cache

class KVCacheManager:
    def __init__(self, model_path: str):
        self.model_path = model_path
        self.identity_cache_path = Path("~/.luna/
        identity_buffer.safetensors")
        self.identity_cache: Optional[KVCache] = None

    async def initialize(self):
        """Load or create identity buffer cache."""
        if self.identity_cache_path.exists():
            self.identity_cache =
        load_kv_cache(self.identity_cache_path)
        else:
            await self.rebuild_identity_cache()

    async def rebuild_identity_cache(self):
        """Rebuild identity buffer KV cache."""
        identity_prompt = await self._load_identity_buffer()

        cache_prompt(
            model=self.model_path,
            prompt=identity_prompt,
```

```
            output_path=str(self.identity_cache_path)
        )

        self.identity_cache =
        load_kv_cache(self.identity_cache_path)

    def get_cache_for_turn(self, conversation_history: str) ->
        KVCache:
        """Get KV cache with identity buffer pinned."""
        # Start with identity buffer (pre-computed)
        cache = self.identity_cache.copy()

        # Extend with conversation history if needed
        # (This part is computed fresh, but identity is free)

        return cache
```

Rolling Cache Strategy

When conversation exceeds context window:

```
class RollingKVCache:
    def __init__(self, max_kv_size: int = 8192):
        self.max_kv_size = max_kv_size
        self.identity_size = 2048  # Pinned, never rotated
        self.available_size = max_kv_size - self.identity_size

    def rotate(self, cache: KVCache, new_tokens: int) -> KVCache:
        """Rotate out old turns while keeping identity pinned."""
        current_size = cache.size()

        if current_size + new_tokens <= self.max_kv_size:
            return cache  # No rotation needed

        # Calculate how much to remove
        remove_tokens = (current_size + new_tokens) -
        self.max_kv_size

        # Remove from middle (after identity, before recent)
        # Identity: [0:2048] — PINNED
        # History:  [2048:X] — Rotate oldest
        # Recent:   [X:end]  — Keep

        return self._rotate_middle(cache, remove_tokens)
```

Cache Warming

Start cache operations before they're needed:

```
async def warm_cache_on_partial(self, partial: str):
    """Pre-warm cache while user is still speaking."""
    # Speculative context assembly
```

```
    speculative_context = await self.assemble_context(partial)

    # Start encoding (but don't block on it)
    self.warm_task = asyncio.create_task(
        self.model.encode(speculative_context)
    )
```

9.4 Vector Index Selection

The Goldilocks Zone

At Luna's scale (~10K nodes), fancy algorithms can actually hurt performance.

Node Count	Recommended Index	Why
<10K	`IndexFlatIP`	Brute force fits in cache
10K-50K	`IndexFlatIP` or `IVF`	Test both
50K+	`HNSW32`	Graph navigation pays off

Why Brute Force Wins at Small Scale

```
IndexFlatIP (Brute Force):
  - 10K vectors × 768 dims × 4 bytes = 30MB
  - Fits entirely in L3/SLC cache on Apple Silicon
  - No graph navigation overhead
  - Exact results (no approximation)
  - ~10-20ms for 10K vectors

HNSW at 10K:
  - Graph structure overhead
  - Navigation overhead per query
  - Often slower than brute force at this scale
  - Approximation errors
```

Implementation

```python
import faiss
import numpy as np

class VectorIndex:
    def __init__(self, dimension: int = 768):
        self.dimension = dimension
        self.index: Optional[faiss.Index] = None
        self.id_map: dict[int, str] = {}  # FAISS ID -> Node ID

    def build(self, vectors: np.ndarray, node_ids: list[str]):
        """Build index with appropriate algorithm."""
        n_vectors = len(vectors)
```

```
        if n_vectors < 10_000:
            # Brute force - fastest at this scale
            self.index = faiss.IndexFlatIP(self.dimension)
        elif n_vectors < 50_000:
            # IVF - good middle ground
            n_clusters = int(np.sqrt(n_vectors))
            quantizer = faiss.IndexFlatIP(self.dimension)
            self.index = faiss.IndexIVFFlat(
                quantizer, self.dimension, n_clusters
            )
            self.index.train(vectors)
        else:
            # HNSW - best for large scale
            self.index = faiss.IndexHNSWFlat(self.dimension, 32)

        # Add vectors
        self.index.add(vectors)

        # Build ID map
        self.id_map = {i: nid for i, nid in enumerate(node_ids)}

    def search(self, query: np.ndarray, k: int = 10) ->
         list[tuple[str, float]]:
        """Search for nearest neighbors."""
        distances, indices = self.index.search(
            query.reshape(1, -1), k
        )

        results = []
        for dist, idx in zip(distances[0], indices[0]):
            if idx >= 0:  # Valid result
                node_id = self.id_map[idx]
                results.append((node_id, float(dist)))

        return results
```

Apple Silicon Optimization

```
# Use float16 for memory efficiency on Apple Silicon
vectors = vectors.astype(np.float16)

# MLX can also do similarity search on GPU
import mlx.core as mx

def mlx_similarity_search(query: mx.array, index: mx.array, k:
         int = 10):
    """GPU-accelerated similarity search using MLX."""
    # Cosine similarity
    similarities = mx.matmul(query, index.T)

    # Top-k
    top_k_indices = mx.argpartition(-similarities, k)[:k]
```

```
    top_k_scores = similarities[top_k_indices]

    return top_k_indices, top_k_scores
```

9.5 Graph Traversal Speed

The Problem

NetworkX is pure Python. Graph operations that should take microseconds take milliseconds.

The Solution: rustworkx

Drop-in replacement with 10-100x speedup:

```
# Before (NetworkX) — ~50ms for 1-hop on 10K nodes
import networkx as nx

def get_neighbors_nx(graph: nx.DiGraph, node_id: str, depth: int
        = 1):
    neighbors = list(nx.bfs_tree(graph, node_id,
        depth_limit=depth))
    return neighbors

# After (rustworkx) — ~0.5ms for same operation
import rustworkx as rx

def get_neighbors_rx(graph: rx.PyDiGraph, node_idx: int, depth:
        int = 1):
    neighbors = rx.bfs_successors(graph, node_idx)
    return [n for d, nodes in neighbors if d <= depth for n in
        nodes]
```

Migration Guide

NetworkX	rustworkx	Notes
`nx.DiGraph()`	`rx.PyDiGraph()`	Same interface
`G.add_node(id)`	`G.add_node(data)`	Returns index
`G.add_edge(u, v)`	`G.add_edge(u_idx, v_idx, data)`	Uses indices
`nx.bfs_tree()`	`rx.bfs_successors()`	Different return format
`nx.shortest_path()`	`rx.dijkstra_shortest_paths()`	Same concept

Hybrid Approach

Keep NetworkX for complex algorithms not in rustworkx, use rustworkx for hot path:

```
class HybridGraph:
    def __init__(self):
        self.rx_graph = rx.PyDiGraph()  # Hot path
        self.nx_graph = nx.DiGraph()     # Complex ops
        self.node_map: dict[str, int] = {}  # ID -> rustworkx
        index

    def add_node(self, node_id: str, data: dict):
        # Add to both
        idx = self.rx_graph.add_node(data)
        self.nx_graph.add_node(node_id, **data)
        self.node_map[node_id] = idx

    def get_neighbors_fast(self, node_id: str, depth: int = 1) ->
        list[str]:
        """Fast neighbor lookup using rustworkx."""
        idx = self.node_map[node_id]
        return rx.bfs_successors(self.rx_graph, idx)

    def get_complex_paths(self, source: str, target: str):
        """Complex pathfinding using NetworkX."""
        return nx.all_simple_paths(self.nx_graph, source, target)
```

9.6 Bloom Filter Pre-check

The Pattern

Before searching, check if the entity even exists. O(1) check can skip O(n) search.

Implementation

```
from pybloom_live import BloomFilter

class EntityFilter:
    def __init__(self, capacity: int = 100_000, error_rate: float
        = 0.001):
        self.filter = BloomFilter(capacity=capacity,
        error_rate=error_rate)

    def add(self, entity: str):
        """Add entity to filter."""
        self.filter.add(entity.lower())

    def might_exist(self, entity: str) -> bool:
```

```python
        """Check if entity might exist (false positives
        possible)."""
        return entity.lower() in self.filter

    def definitely_missing(self, entity: str) -> bool:
        """Check if entity definitely doesn't exist (no false
        negatives)."""
        return entity.lower() not in self.filter

class OptimizedRetrieval:
    def __init__(self, matrix: MemoryMatrix, entity_filter:
        EntityFilter):
        self.matrix = matrix
        self.entity_filter = entity_filter

    async def search(self, query: str) -> list[Node]:
        """Search with Bloom filter pre-check."""
        # Extract entities from query
        entities = extract_entities(query)

        # Check which entities might exist
        possible_entities = [
            e for e in entities
            if self.entity_filter.might_exist(e)
        ]

        if not possible_entities:
            # No entities exist, skip graph traversal
            # Fall back to pure semantic search
            return await self.matrix.semantic_search(query)

        # Entities might exist, do full search
        return await self.matrix.hybrid_search(query,
        entities=possible_entities)
```

When to Use

Scenario	Bloom Filter Helps?
"What did we decide about X?"	Yes — check if X exists
"How are you feeling?"	No — no entity to check
"Tell me about Project Luna"	Yes — check if "Project Luna" exists
Conceptual/vague queries	No — use semantic search directly

9.7 Query Routing

The Problem

Running every search path every time wastes cycles: - FTS5 for keywords - FAISS for semantics - Graph for relationships

Most queries only need one path.

Heuristic Router

```python
from enum import Enum

class SearchPath(Enum):
    FTS5 = "fts5"
    FAISS = "faiss"
    GRAPH = "graph"
    HYBRID = "hybrid"

class QueryRouter:
    def __init__(self):
        self.entity_extractor = EntityExtractor()

    def route(self, query: str) -> SearchPath:
        """Determine optimal search path."""

        # Extract features
        entities = self.entity_extractor.extract(query)
        keyword_density = self._keyword_density(query)
        is_conceptual = self._is_conceptual(query)

        # Routing logic
        if entities and len(entities) >= 2:
            # Multiple entities - relationship query
            return SearchPath.GRAPH

        if keyword_density > 0.6:
            # High keyword density - exact match
            return SearchPath.FTS5

        if is_conceptual:
            # Conceptual/vague - semantic search
            return SearchPath.FAISS

        # Default to hybrid
        return SearchPath.HYBRID

    def _keyword_density(self, query: str) -> float:
        """Ratio of content words to total words."""
        words = query.split()
```

```
        stopwords = {"the", "a", "an", "is", "are", "what",
        "how", "did", "we"}
        content_words = [w for w in words if w.lower() not in
        stopwords]
        return len(content_words) / len(words) if words else 0

    def _is_conceptual(self, query: str) -> bool:
        """Check if query is conceptual/abstract."""
        conceptual_markers = [
            "how does", "what is", "explain", "describe",
            "concept", "idea", "theory", "approach"
        ]
        return any(marker in query.lower() for marker in
        conceptual_markers)
```

Tiny Model Router (Optional Upgrade)

For more nuanced routing, use a small classifier:

```
class TinyModelRouter:
    def __init__(self, model_path: str = "smollm-135m"):
        self.model = load_model(model_path)
        self.prompt_template = """Classify this query's search
        type:
Query: {query}

Types:
- KEYWORD: Looking for specific terms/names
- SEMANTIC: Conceptual or vague
- RELATIONSHIP: Asking about connections between things
- HYBRID: Needs multiple approaches

Type:"""

    async def route(self, query: str) -> SearchPath:
        prompt = self.prompt_template.format(query=query)
        response = await self.model.generate(prompt,
        max_tokens=10)

        # Parse response
        response_lower = response.lower()
        if "keyword" in response_lower:
            return SearchPath.FTS5
        elif "semantic" in response_lower:
            return SearchPath.FAISS
        elif "relationship" in response_lower:
            return SearchPath.GRAPH
        else:
            return SearchPath.HYBRID
```

9.8 Reciprocal Rank Fusion (RRF)

The Problem

When you run multiple search paths, how do you merge results?

Naive approaches fail: - Raw score combination (scales differ) - Weighted average (need to tune weights) - Union (no ranking)

The Solution: RRF

Documents appearing in multiple result lists rank higher. Battle-tested at Google and Perplexity.

Formula

```
RRF_score(doc) = Σ (1 / (k + rank_in_list))

where k = 60 (tunable constant)
```

Implementation

```python
def rrf_merge(
    result_lists: list[list[str]],
    k: int = 60
) -> list[str]:
    """
    Merge multiple ranked result lists using RRF.

    Args:
        result_lists: List of ranked document ID lists
        k: Ranking constant (default 60)

    Returns:
        Merged, re-ranked list of document IDs
    """
    scores: dict[str, float] = {}

    for result_list in result_lists:
        for rank, doc_id in enumerate(result_list):
            if doc_id not in scores:
                scores[doc_id] = 0.0
            scores[doc_id] += 1.0 / (k + rank)

    # Sort by score descending
    ranked = sorted(scores.keys(), key=lambda d: scores[d],
        reverse=True)

    return ranked
```

```python
# Usage
fts5_results = ["doc1", "doc3", "doc5", "doc7"]
faiss_results = ["doc2", "doc1", "doc4", "doc3"]

merged = rrf_merge([fts5_results, faiss_results])
# Result: ["doc1", "doc3", "doc2", "doc5", "doc4", "doc7"]
# doc1 and doc3 rank highest because they appear in both lists
```

With Scores

```python
def rrf_merge_with_scores(
    result_lists: list[list[tuple[str, float]]],
    k: int = 60
) -> list[tuple[str, float]]:
    """RRF merge preserving scores for downstream use."""
    scores: dict[str, float] = {}
    original_scores: dict[str, list[float]] = {}

    for result_list in result_lists:
        for rank, (doc_id, score) in enumerate(result_list):
            if doc_id not in scores:
                scores[doc_id] = 0.0
                original_scores[doc_id] = []
            scores[doc_id] += 1.0 / (k + rank)
            original_scores[doc_id].append(score)

    # Sort by RRF score
    ranked = sorted(scores.keys(), key=lambda d: scores[d],
        reverse=True)

    # Return with RRF scores
    return [(doc_id, scores[doc_id]) for doc_id in ranked]
```

9.9 Parallel Prefetching

The Pattern

While the Director generates tokens, prefetch context for the next turn.

Implementation

```python
class PrefetchingRetriever:
    def __init__(self, matrix: MatrixActor):
        self.matrix = matrix
        self.prefetch_cache: dict[str, list[Node]] = {}

    async def retrieve_with_prefetch(
        self,
        query: str,
```

```
        active_entities: list[str]
    ) -> list[Node]:
        """Retrieve for current query, prefetch for next."""

        # Check prefetch cache
        cache_key = self._cache_key(query)
        if cache_key in self.prefetch_cache:
            results = self.prefetch_cache.pop(cache_key)
        else:
            results = await self.matrix.search(query)

        # Start prefetch for likely next queries
        asyncio.create_task(
            self._prefetch_neighbors(active_entities)
        )

        return results

    async def _prefetch_neighbors(self, entities: list[str]):
        """Prefetch 1-hop neighbors of active entities."""
        for entity in entities[:3]:  # Limit to top 3
            neighbors = await self.matrix.get_neighbors(entity,
        depth=1)

            # Cache for potential next query
            for neighbor in neighbors:
                cache_key = self._cache_key(neighbor.content[:50])
                self.prefetch_cache[cache_key] = [neighbor]

        # Prune old cache entries
        if len(self.prefetch_cache) > 100:
            # Remove oldest half
            keys = list(self.prefetch_cache.keys())
            for key in keys[:50]:
                del self.prefetch_cache[key]

    def _cache_key(self, text: str) -> str:
        """Generate cache key from text."""
        return hashlib.md5(text.encode()).hexdigest()[:16]
```

When to Prefetch

```
class DirectorActor(Actor):
    async def generate_with_prefetch(self, prompt: str, context:
        list[Node]):
        """Generate response while prefetching."""

        # Extract entities from current context
        active_entities = self._extract_entities(context)

        # Start prefetch task
        prefetch_task = asyncio.create_task(
```

```python
            self.retriever.prefetch_neighbors(active_entities)
        )

        # Generate response (doesn't wait for prefetch)
        async for token in self.model.generate_stream(prompt):
            yield token

        # Prefetch completes in background
        # Results cached for next turn
```

9.10 The Complete Optimized Pipeline

Flow Diagram

```
User starts speaking (t=0ms)
        │
        ├──► [PARALLEL] Pre-warm KV cache
        │          └── Load identity_buffer.safetensors (~5ms)
        │
        ├──► [PARALLEL] Speculative retrieval (debounced)
        │          │
        │          ├── Bloom filter: entities exist? (~1ms)
        │          │
        │          ├── Query router: which path? (~2ms)
        │          │          │
        │          │          ├── KEYWORD → FTS5 only
        │          │          ├── SEMANTIC → FAISS only
        │          │          ├── RELATIONSHIP → Graph only
        │          │          └── HYBRID → FTS5 + FAISS + RRF
        │          │
        │          ├── Execute search (~30ms)
        │          │
        │          └── rustworkx graph hop (~5ms)
        │
User finishes speaking (t≈800ms)
        │
        ├──► Final STT transcript
        │
        ├──► Validate speculation (~5ms)
        │          │
        │          ├── Cosine similarity > 0.85? → Keep results
        │          └── Otherwise → Fetch delta + RRF merge
        │
        ├──► Context assembly (~10ms)
        │          └── Format with <memory> tags
        │
        └──► Director inference
                   │
                   ├── KV cache already warm
                   ├── Context already encoded (if speculative)
```

```
                    │
                    └──► First token (~250ms from final transcript)
                                │
                                └──► TTS (~40ms to audio)

TOTAL: <500ms from end of speech to first audio ✓
```

Configuration

```python
@dataclass
class PerformanceConfig:
    # Speculative execution
    speculation_enabled: bool = True
    speculation_debounce_ms: int = 200
    speculation_threshold: float = 0.85

    # KV cache
    kv_cache_enabled: bool = True
    identity_buffer_size: int = 2048
    max_kv_size: int = 8192

    # Vector index
    vector_index_type: str = "flat"  # "flat", "ivf", "hnsw"
    vector_dimension: int = 768

    # Graph
    graph_backend: str = "rustworkx"  # "rustworkx", "networkx"
    max_graph_depth: int = 2

    # Bloom filter
    bloom_enabled: bool = True
    bloom_capacity: int = 100_000
    bloom_error_rate: float = 0.001

    # Query routing
    router_type: str = "heuristic"  # "heuristic", "model"
    router_model: Optional[str] = None  # "smollm-135m" if model

    # RRF
    rrf_k: int = 60

    # Prefetch
    prefetch_enabled: bool = True
    prefetch_cache_size: int = 100
```

Summary

Performance optimizations that hit the <500ms target:

Optimization	Savings	Priority
Speculative execution	~50ms (retrieval hidden)	🔴 HIGH
KV cache pinning	~50-100ms	🔴 HIGH
rustworkx	~45ms	🔴 HIGH
Query routing	~20-30ms (skip paths)	🟡 MEDIUM
Bloom filter	~5-10ms (skip search)	🟡 MEDIUM
RRF merge	Better results, same time	🟡 MEDIUM
Parallel prefetch	Amortized, invisible	🟢 LOW

Constitutional Principle: Luna should feel like she "already knows" — not "let me look that up."

Next Section: Part X — Sovereignty Infrastructure